

머신러닝 평가지표 해석 가이드

목차

1. 평가지표 한눈에 보기 — 두 종류의 질문
2. 출발점: 정확도의 함정 (MNIST 가짜 분류기 실습)
3. 오차 행렬(Confusion Matrix) 해석법
4. 정밀도(Precision)·재현율(Recall) 해석법
5. F1 스코어 해석법 — 왜 조화평균인가
6. 예측 확률(predict_proba)과 임계값 — 점수는 어디서 오는가
7. ROC 곡선과 AUC 해석법 — 예측 확률이 얼마나 구분을 잘하는가
8. 단계별 해석 루틴 (실전 5단계)
9. 사례별 해석 가이드 (6가지 상황)
10. 핵심 API 요약표
11. 머신러닝 활용 포인트

1. 평가지표 한눈에 보기 — 두 종류의 질문

분류 모델의 예측 과정은 두 단계입니다. 먼저 각 데이터에 대해 **예측 확률(점수)**을 계산하고(`predict_proba`), 그 점수를 임계값(기본 0.5)과 비교해 **최종 0/1 판정**을 내립니다(`predict`). 평가지표도 이 두 단계에 각각 대응하는 두 그룹으로 나뉩니다.

평가 대상	묻는 질문	해당 지표	임계값 영향
최종 0/1 판정	"분류를 잘 했는가" — 판정이 정답과 얼마나 일치했나	정확도, 오차 행렬, 정밀도, 재현율, F1	임계값에 따라 값이 변함
예측 확률(점수)	"잘 가려냈는가" — 점수가 두 클래스를 얼마나 구분하는가	ROC-AUC (보조: PR-AUC)	임계값과 무관

■ 핵심 구도

AUC는 "분류를 잘할 수 있는 잠재력(체급)", 정밀도·재현율·F1은 "실제로 분류한 성적표"입니다. AUC가 높는데 F1이 낮으면 모델이 아니라 임계값이 문제일 가능성이 크고, AUC 자체가 낮으면 임계값을 어디에 뒀도 해결되지 않습니다. 이 구분이 본 자료 전체를 관통하는 핵심 구도이며, 7장과 9장에서 사례로 확인합니다.

2. 출발점: 정확도의 함정 (MNIST 가짜 분류기 실습)

왜 정확도 하나로는 부족한지, 학습을 전혀 하지 않는 "가짜 분류기"로 확인합니다. MNIST 손글씨 데이터에서 숫자 7이면 1, 아니면 0인 이진 분류 문제를 만들고, **무조건 0만 출력하는** 분류기를 평가합니다.

[코드]

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.base import BaseEstimator
from sklearn.metrics import accuracy_score
import numpy as np
import pandas as pd

class MyFakeClassifier(BaseEstimator):
    def fit(self, X, y):
        pass
    def predict(self, X):
        return np.zeros((len(X), 1), dtype=bool)

digits = load_digits()
y = (digits.target == 7).astype(int)
X_train, X_test, y_train, y_test = train_test_split(digits.data, y, random_state=11)

print('레이블 테스트 세트 크기 :', y_test.shape)
print(pd.Series(y_test).value_counts())

fakeclf = MyFakeClassifier()
fakeclf.fit(X_train, y_train)
```

```
fakepred = fakeclf.predict(X_test)
print('모든 예측을 0으로 하여도 정확도는: {:.3f}'.format(accuracy_score(y_test, fakepred)))
```

줄	코드	상세 설명
8	<code>class MyFakeClassifier(BaseEstimator):</code>	<code>BaseEstimator</code> 를 상속해 사이킷런 규약(<code>fit/predict</code>)을 따르는 커스텀 분류기 정의. 사이킷런의 다른 도구(교차검증 등)와 호환됨
9~10	<code>def fit(self, X, y): pass</code>	학습을 전혀 하지 않음 — 의도적으로 만든 "가짜" 학습
11~12	<code>def predict(self, X): return np.zeros((len(X), 1), dtype=bool)</code>	입력 데이터 건수만큼 전부 0(False)으로 채운 배열 반환. 즉 모든 데이터를 무조건 "7이 아님"으로 판정
15	<code>y = (digits.target == 7).astype(int)</code>	타깃이 7이면 True→1, 아니면 False→0으로 변환. 10개 클래스 문제를 이진 분류로 바꿈. 7은 전체의 약 10%뿐이므로 의도적으로 불균형한 레이블이 됨
16	<code>train_test_split(digits.data, y, random_state=11)</code>	기본 비율(<code>test_size=0.25</code>)로 학습/테스트 분리. <code>random_state=11</code> 로 재현성 고정
19	<code>pd.Series(y_test).value_counts()</code>	테스트 세트의 0/1 분포 확인 — 불균형 정도를 눈으로 확인하는 단계
24	<code>accuracy_score(y_test, fakepred)</code>	정확도 = (맞힌 건수)/(전체 건수). 인자 순서는 (실제, 예측)

[실행 결과]

```
레이블 테스트 세트 크기 : (450,)
0    405
1     45
Name: count, dtype: int64
모든 예측을 0으로 하여도 정확도는:0.900
```

■ 결과 해석

아무것도 학습하지 않고 전부 0으로 찍는 분류기가 정확도 **90%**를 받았습니다. 테스트 450건 중 0이 405건(90%)인 불균형 데이터이기 때문입니다. 불균형 데이터에서 정확도는 "다수 클래스로 전부 찍기"라는 꼼수를 걸러내지 못합니다. 그래서 오차 행렬부터 시작하는 다른 지표들이 필요합니다.

3. 오차 행렬(Confusion Matrix) 해석법

[코드]

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, fakepred)
```

줄	코드	상세 설명
1	<code>from sklearn.metrics import confusion_matrix</code>	평가 지표 모듈에서 오차 행렬 함수 импорт
2	<code>confusion_matrix(y_test, fakepred)</code>	첫 번째 인자 = 실제 , 두 번째 인자 = 예측 . 순서가 바뀌면 FN·FP 위치가 뒤바뀌므로 반드시 (실제, 예측) 순서 준수

[실행 결과] (scikit-learn 1.7.2)

```
array([[405,  0],
       [ 45,  0]])
```

3.1 구조 읽기 — 행은 실제, 열은 예측

사이킷런의 오차 행렬은 **행(row) = 실제 클래스**, **열(column) = 예측 클래스**이며 레이블을 오름차순(0→1)으로 배치합니다.

	예측: 0 (Negative)	예측: 1 (Positive)
실제: 0 (Negative)	TN (True Negative) = 405	FP (False Positive) = 0
실제: 1 (Positive)	FN (False Negative) = 45	TP (True Positive) = 0

기호 읽는 법 — 앞 글자(T/F) = 예측이 맞았는가, 뒷 글자(N/P) = 무엇으로 예측했는가:

- **TN = 405** : 실제 7이 아닌데(0), 7이 아니라고(0) 맞게 예측
- **FP = 0** : 실제 7이 아닌데(0), 7이라고(1) 잘못 예측 — **오탐**
- **FN = 45** : 실제 7인데(1), 7이 아니라고(0) 잘못 예측 — **놓침**
- **TP = 0** : 실제 7인데(1), 7이라고(1) 맞게 예측

3.2 해석 포인트

■ 결과 해석

예측이 1인 오른쪽 열이 전부 0 — 이 분류기는 Positive 예측을 단 한 번도 하지 않았습니다. 그런데도 정확도는 $(TN+TP)/\text{전체} = 405/450 = 0.900$. 오차 행렬을 보면 "정확도 90%"의 실체가 "다수 클래스만 맞히고 소수 클래스는 전멸(FN=45, TP=0)"임이 즉시 드러납니다. **평가의 첫 단계는 항상 오차 행렬로 4개 칸의 절대 건수를 확인하는 것**입니다.

⚠ 주의

FP와 FN은 업무적 비용이 다릅니다. 암 진단에서 FN(환자를 정상 판정)은 생명과 직결되지만 FP(정상인을 재검사)는 비용 문제에 그칩니다. 어느 칸의 오류가 치명적인지는 지표가 아니라 **업무 요구사항**이 결정하며, 이것이 4장에서 정밀도/재현율 중 무엇을 우선할지의 기준이 됩니다.

4. 정밀도(Precision)·재현율(Recall) 해석법

4.1 공식과 직관 — 분자는 같고 분모가 다르다

지표	공식	분모의 의미	물어 질문
정밀도	$TP / (FP + TP)$	모델이 Positive라고 예측한 전체 (오차 행렬의 오른쪽 열 합)	"모델이 7이라고 한 것 중 실제 7의 비율" — 양성 판정의 신뢰도. FP를 낮추는데 초점

재현율	TP / (FN + TP)	실제 Positive 전체 (오차 행렬의 아래 행 합)	"실제 7 중 모델이 찾아낸 비율" — 탐지 능력. FN을 낮추는 데 초점
-----	----------------	--------------------------------	---

■ 암기 포인트

두 지표 모두 분자는 TP로 같습니다. 정밀도는 "**예측 기준**"으로, 재현율은 "**실제 기준**"으로 나눕니다. 용어의 어원도 도움이 됩니다 — 둘 다 정보 검색 분야에서 왔습니다. Recall은 "존재하는 관련 문서를 얼마나 다시 불러왔는가(재현)", Precision은 "가져온 결과가 얼마나 불순물 없이 정밀한가(순도)"입니다. 재현율의 별칭 민감도(Sensitivity)는 의학에서 같은 값을 부르는 이름입니다.

4.2 가짜 분류기에 적용

[코드]

```
from sklearn.metrics import precision_score, recall_score
print("정밀도:", precision_score(y_test, fakepred))
print("재현율:", recall_score(y_test, fakepred))
```

[실행 결과] (scikit-learn 1.7.2)

```
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
due to no predicted samples. Use `zero_division` parameter to control this behavior.
정밀도: 0.0
재현율: 0.0
```

오차 행렬 값(TN=405, FP=0, FN=45, TP=0)을 대입해 계산하면:

- **정밀도** = $0/(0+0) = 0/0 \rightarrow$ 정의 불가. Positive 예측이 한 건도 없어 분모가 0. 사이킷런은 경고를 띄우고 0.0을 반환
- **재현율** = $0/(45+0) = 0.0$. 실제 7이 45개 있는데 하나도 못 찾음

■ 결과 해석

정확도 0.900짜리 모델의 정밀도·재현율이 모두 0입니다. 불균형 데이터에서 정확도가 놓치는 것을 이 두 지표가 정확히 잡아냅니다.

⚠ 버전·파라미터 주의

0/0 상황의 처리는 zero_division 파라미터로 제어합니다(scikit-learn 0.22 이상). precision_score(y_test, pred, zero_division=0)처럼 명시하면 경고 없이 0을 반환하고, zero_division=1이면 1을 반환합니다.

4.3 해석 시 체크 포인트

- **정밀도만 높고 재현율이 매우 낮으면** — 확실한 것만 골라 Positive로 예측하는 "소극적" 모델. 극단적으로 1건만 예측해 맞으면 정밀도 1.0이 되므로 정밀도 단독 평가 금물
- **재현율만 높고 정밀도가 매우 낮으면** — 전부 Positive로 찍는 "남발형" 모델. 전부 Positive로 예측하면 재현율 1.0이 되므로 역시 단독 평가 금물
- 따라서 두 지표는 **항상 쌍으로** 읽고, 균형은 5장의 F1로 확인

4.4 어느 지표를 우선할 것인가 — 업무별 기준

우선 지표	기준	대표 사례
재현율 우선	실제 Positive를 놓치는 것(FN)의 비용이 큰 업무	암 진단(환자를 정상 판정하면 치명적), 금융 사기 탐지(사기 거래를 놓치면 직접 손실)
정밀도 우선	Negative를 Positive로 잘못 판정(FP)하는 비용이 큰 업무	스팸 필터(업무 메일을 스팸 처리하면 중요 메일 유실, 스팸을 놓치는 건 불편에 그침)

4.5 트레이드오프

정밀도와 재현율은 분류 결정 임계값을 매개로 서로 반대로 움직입니다. 임계값을 **낮추면** Positive 판정이 많아져 재현율 ↑ 정밀도 ↓, **높이면** 정밀도 ↑ 재현율 ↓. 임계값과 점수의 관계는 6장에서, 임계값별 변화의 전체 그림은 7장의 ROC 곡선에서 다룹니다.

5. F1 스코어 해석법 — 왜 조화평균인가

$$F1 = 2 \times (\text{정밀도} \times \text{재현율}) / (\text{정밀도} + \text{재현율})$$

F1은 정밀도와 재현율의 **조화평균**입니다. 산술평균이 아닌 조화평균을 쓰는 이유는 **낮은 쪽 값에 크게 끌려가는** 성질 때문입니다. 한쪽이 0에 가까우면 F1도 0에 가까워져, 한 지표만 극단적으로 높인 "꿈수 모델"에 낮은 점수를 줍니다.

정밀도	재현율	산술평균	F1 (조화평균)	해석
0.9	0.1	0.50	0.18	극단 불균형 — 산술평균은 후하게, F1은 박하게 평가
0.5	0.5	0.50	0.50	균형 상태 — 두 평균이 일치
0.8302	0.6769	0.754	0.7458	7장 타이타닉 모델 — 약간의 불균형만큼 산술평균보다 낮음

해석 시 체크 포인트

- F1은 **임계값에 종속**된 지표 — 임계값을 바꾸면 F1도 바뀝니다. 모델 간 비교 시 같은 임계값 기준인지 확인
- F1이 같아도 (정밀도 0.9, 재현율 0.6)과 (0.6, 0.9)는 전혀 다른 모델 — **F1만 보지 말고 항상 구성 요소를 함께** 볼 것
- 산술평균과 F1의 차이가 클수록 정밀도·재현율의 불균형이 심하다는 신호
- 불균형 데이터에서 정확도를 대신하는 대표 지표

6. 예측 확률(predict_proba)과 임계값 — 점수는 어디서 오는가

6장부터는 타이타닉 생존 예측 데이터에 로지스틱 회귀를 적용한 결과로 설명합니다. 정밀도·재현율·F1이 "판정"의 성적표라면, 판정의 재료인 **점수(예측 확률)**가 무엇인지 이해해야 7장의 AUC를 해석할 수 있습니다.

⚠ 실행 환경 주의

본 자료의 타이타닉 실행 결과는 확보 가능한 데이터 839건 기준입니다(scikit-learn 1.7.2, LogisticRegression(solver='liblinear'), test_size=0.20, random_state=11, 테스트 168건). 원본 캐글 데이터(891건, 테스트 179건)로 실행하면 확률의 구체적 숫자는 달라지지만 **구조와 해석 방법은 동일합니다**.

6.1 predict_proba — 확률을 먼저, 판정은 그 다음

[코드]

```
pred_proba = lr_clf.predict_proba(X_test)
pred = lr_clf.predict(X_test)
print('pred_proba()결과 Shape : {0}'.format(pred_proba.shape))
print('pred_proba array에서 앞 3개만 샘플로 추출 \n:', pred_proba[:3])

# 예측 확률과 예측 결과값을 옆으로 붙여 한눈에 확인
pred_proba_result = np.concatenate([pred_proba, pred.reshape(-1,1)], axis=1)
print('두개의 class 중에서 더 큰 확률을 클래스 값으로 예측 \n', pred_proba_result[:3])
```

줄	코드	상세 설명
1	<code>lr_clf.predict_proba(X_test)</code>	각 건에 대한 클래스별 예측 확률 . 이진 분류이므로 (건수, 2) 배열 — 1열 = 클래스 0(사망) 확률, 2열 = 클래스 1(생존) 확률. 두 열의 합은 항상 1. 열 순서는 <code>lr_clf.classes_</code> (오름차순) 순서
2	<code>lr_clf.predict(X_test)</code>	최종 0/1 판정. 내부적으로 <code>predict_proba</code> 에서 확률이 더 큰 열을 고르는 것과 동일(이진 분류에선 Positive 확률 ≥ 0.5 여부)
7	<code>pred.reshape(-1,1)</code>	1차원 (n,) 배열을 2차원 (n,1)로 변환. -1은 "행 수는 자동 계산". 2차원인 <code>pred_proba</code> 와 옆으로 붙이기 위한 사전 작업
7	<code>np.concatenate(..., axis=1)</code>	<code>axis=1</code> (열 방향)로 이어붙여 (n,3) 배열 생성 — 각 행이 [0 확률, 1 확률, 최종 판정]

[실행 결과]

```
pred_proba()결과 Shape : (168, 2)
pred_proba array에서 앞 3개만 샘플로 추출
: [[0.69940875 0.30059125]
   [0.92355652 0.07644348]
   [0.79577918 0.20422082]]
두개의 class 중에서 더 큰 확률을 클래스 값으로 예측
[[0.69940875 0.30059125 0.
  [0.92355652 0.07644348 0.
  [0.79577918 0.20422082 0.]]
```

■ 결과 해석

1행: 사망 확률 0.699 vs 생존 확률 0.301 → 큰 쪽인 0(사망)이 3열의 최종 판정과 일치. `predict()`는 별도 알고리즘이 아니라 `predict_proba()` 위에 "Positive 확률 ≥ 0.5 면 1"이라는 결정 규칙을 얹은 것입니다. **0.5는 고정된 진리가 아니라 조정 가능한 값** — 이것이 임계값 튜닝의 출발점입니다.

6.2 점수의 정체 — 손으로 재현 가능한 산수

0.30059라는 점수는 신비로운 값이 아닙니다. 로지스틱 회귀는 학습이 끝나면 피처별 가중치 w 와 절편 b 를 가지며, 새 데이터에 대해 두 단계만 계산합니다.

[실행 결과 — 학습된 모델 내부]

```

피처:      [Pclass, Sex,      Age,      SibSp,  Parch,  Fare,  Cabin,  Embarked]
가중치 w: [-0.7027, -2.4249, -0.0244, -0.2924, -0.1448, 0.0039, -0.0714, -0.0054]
절편 b:    3.795

첫 번째 테스트 승객: Pclass=1, Sex=1(남성), Age=45, Fare=26.55, ...
1단계   $z = w \cdot x + b = (-0.7027 \times 1) + (-2.4249 \times 1) + (-0.0244 \times 45) + \dots + 3.795 = -0.8445$ 
2단계  시그모이드  $1/(1+e^{-z}) = 0.30059$ 
       predict_proba 결과      = 0.30059 ← 정확히 일치
  
```

- **1단계 (점수 합산):** $z = w_1 \times \text{피처}_1 + w_2 \times \text{피처}_2 + \dots + b$. 피처값과 학습된 가중치의 곱의 합
- **2단계 (시그모이드 변환):** z 는 $\pm\infty$ 어떤 값이든 될 수 있으므로 $1/(1+e^{-z})$ 로 0~1 사이에 눌러 담아 확률 형태로 만들

■ 왜 이런 점수를 주는가

가중치는 훈련 데이터의 통계입니다. Sex 가중치가 -2.42로 압도적인 이유는 훈련 데이터에서 "남성(=1)은 생존율이 낮다"는 패턴이 강했기 때문입니다. 즉 이 승객의 0.30점은 "당신과 비슷한 조건(중년 남성 등)의 승객들이 훈련 데이터에서 대략 이 정도 비율로 생존했다"를 공식 하나로 압축한 값입니다.

⚠ 주의 — 분류기는 비교하지 않는다

분류기는 한 건씩 독립적으로 점수를 계산합니다. 다른 데이터와 비교·정렬하는 동작은 분류기 내부에 없습니다. 7장에서 등장하는 "정렬·순위"는 분류기의 동작이 아니라 평가자가 점수들을 모아 놓고 보는 관점이라는 점을 미리 구분해 두세요.

6.3 이 모델의 판정 성적표 (임계값 0.5 기준)

[실행 결과]

```

오차행렬: [[94  9]      정확도: 0.8214   정밀도: 0.8302
           [21 44]]     재현율: 0.6769   F1: 0.7458   AUC: 0.8513
  
```

읽는 연습 (TN=94, FP=9, FN=21, TP=44):

- **정밀도 0.8302** = $44/(9+44)$ — "생존 판정 53명 중 44명이 실제 생존자". 생존 판정은 83% 신뢰 가능
- **재현율 0.6769** = $44/(21+44)$ — "실제 생존자 65명 중 44명 탐지". 32%(21명)를 놓침(FN)
- **F1 0.7458** — 정밀도 쪽으로 기운 모델. "생존 판정은 신중하지만 그만큼 놓친다"
- **AUC 0.8513** — 7장에서 해석

7. ROC 곡선과 AUC 해석법 — 예측 확률이 얼마나 구분을 잘하는가

■ 한 줄 정의 (이것부터)

AUC = 예측 확률이 두 클래스를 얼마나 잘 구분하는가. 1.0이면 완벽한 구분, 0.5면 구분 못함(무작위와 동일). 이 한 문장이 핵심이고, 아래는 전부 이 문장의 각주입니다.

7.1 "구분"의 뜻 — 점수·정답 정렬표로 확인

테스트 168명 각각의 점수(모델이 독립적으로 계산)와 실제 정답(1=생존, 0=사망)을 **평가자가** 점수 높은 순으로 늘어놓은 표입니다. "구분이 잘된다"는 것은 이 표의 **실제정답 열의 모양**을 말합니다.

[실행 결과 — 점수 내림차순 정렬표 발췌]

--- 상위 구간: 실제정답이 거의 다 1 ---				--- 하위 구간: 전부 0 ---			
점수	실제정답			점수	실제정답		
0.965	1			0.078	0		
0.947	0	← 홀로 끼어든 예외		0.069	0		
0.938	1			0.048	0		
0.937	1			0.013	0		
--- 중간 구간(0.3~0.6): 1과 0이 교대로 등장 = "교대 구간" ---							
0.579	0	0.556	0	0.511	1	0.472	0
0.576	0	0.555	1	0.475	1	0.425	0
0.562	1	0.523	1			0.406	1

중간 구간에서는 0.556점 받은 사람은 죽었는데 0.555점 받은 사람은 살았습니다 — **점수가 정답과 따로 노는 구간**입니다. 이 구간 안에서는 어떤 점수로 잘라도 두 클래스를 가릴 수 없습니다.

■ AUC는 결국 "교대 구간이 얼마나 좁은가"

위쪽은 1들, 아래쪽은 0들로 깔끔히 나뉘고 교대 구간이 없으면 $AUC = 1.0$. 표 전체가 교대 구간이면 $AUC \approx 0.5$. 이 모델의 0.8513은 "위아래는 잘 나뉘었지만 중간에 교대 구간이 좀 있다"의 숫자 요약입니다. 두 가지가 함께 반영됩니다 — (1) 교대 구간의 폭과 그 안의 교대 **심각도**(1 1 1 0 0 0에 가까우면 덜 짝이고 1 0 1 0이면 많이 짝임), (2) 상위 권에 홀로 끼어든 0.947점 사망자 같은 **돌출 사례**(자기보다 아래의 생존자 60여 명과 전부 뒤집힌 쌍을 만들어 크게 감점).

7.2 계산 방법 — 쌍 비교 (수학적 정의)

구분 정도를 숫자 하나로 만드는 방법이 (**실제 1, 실제 0**) **쌍 비교**입니다. 실제 생존자 1명과 실제 사망자 1명을 뽑아 **이미 매겨져 있던 점수**를 비교했을 때 생존자 쪽이 높으면 "승". 모든 쌍에 대한 승률이 AUC입니다. (상승-상승, 사망-사망 쌍은 구분 능력과 무관하므로 세지 않습니다.)

[검증 코드]

```
pos = proba[y_test == 1]          # 실제 생존자 65명의 점수
neg = proba[y_test == 0]          # 실제 사망자 103명의 점수
diff = pos.reshape(-1,1) - neg.reshape(1,-1) # 65×103 = 6,695개 모든 쌍의 차이
manual_auc = ((diff > 0).sum() + 0.5*(diff == 0).sum()) / diff.size
```

[실행 결과]

생존자 확률이 더 높은 쌍: 5699 / 동점: 1 (동점은 0.5승 처리)
 쌍 비교로 계산한 AUC : 0.8513
 roc_auc_score : 0.8513 ← 소수 4자리까지 일치
 무작위 추출 10만회 시뮬레이션: 0.8514

■ 결과 해석

$5699.5/6695 = 0.8513$ — ROC 곡선 아래 면적과 쌍 비교 승률은 **같은 값의 두 가지 계산법**입니다(통계학의 Mann-Whitney U 통계량과 동일). 뒤집힌 쌍 하나하나가 감점이며, "뒤집힌 쌍의 개수"는 임계값을 어떻게 고르든 피할 수 없는 구조적 오류의 총량입니다.

7.3 roc_curve — 임계값을 훑으며 곡선 그리기

[코드]

```
from sklearn.metrics import roc_curve

# 레이블 값이 1일 때의 예측 확률 추출
pred_proba_class1 = lr_clf.predict_proba(X_test)[: , 1]

fprs, tprs, thresholds = roc_curve(y_test, pred_proba_class1)
thr_index = np.arange(1, thresholds.shape[0], 5)
print('샘플 index로 추출한 임계값: ', np.round(thresholds[thr_index], 2))
print('샘플 임계값별 FPR: ', np.round(fprs[thr_index], 3))
print('샘플 임계값별 TPR: ', np.round(tprs[thr_index], 3))
```

줄	코드	상세 설명
4	<code>predict_proba(X_test)[: , 1]</code>	(n,2) 확률 배열에서 2열(Positive=1 확률)만 추출. roc_curve 는 Positive 확률 1차원 배열 을 요구
6	<code>roc_curve(y_test, pred_proba_class1)</code>	임계값을 최고 확률부터 낮춰가며 각 임계값마다 오차 행렬을 다시 계산 한 결과를 같은 길이의 배열 3개로 반환. 같은 인덱스끼리 한 세트: 임계값 <code>thresholds[i]</code> 로 자르면 성적이 (<code>fprs[i]</code> , <code>tprs[i]</code>)
7	<code>np.arange(1, thresholds.shape[0], 5)</code>	인덱스 1부터 5칸 간격 샘플링(전체 출력이 길어서). 0이 아닌 1부터 시작하는 이유는 아래 버전 주의 참조

[실행 결과] (배열 길이 56)

샘플 index로 추출한 임계값: [0.96 0.86 0.66 0.6 0.43 0.31 0.26 0.2 0.15 0.12 0.11]
 샘플 임계값별 FPR: [0. 0.019 0.039 0.058 0.107 0.184 0.252 0.35 0.476 0.709 0.796]
 샘플 임계값별 TPR: [0.015 0.2 0.492 0.615 0.692 0.769 0.815 0.846 0.908 0.923 0.985]

tprs(TPR) = TP/(FN+TP) — 재현율과 동일. ROC 곡선의 y축. **fprs(FPR)** = FP/(FP+TN) — 실제 Negative 중 오탐 비율. x축. 세로로 한 열씩 읽습니다:

임계값	FPR	TPR	해석
0.96	0.000	0.015	확률 96% 이상만 생존 판정 → 오탐 0이지만 탐지도 1.5%뿐 (극도로 보수적)

0.60	0.058	0.615	오탐 5.8%를 감수하고 생존자 61.5% 탐지
0.43	0.107	0.692	임계값 0.6→0.43: TPR +7.7%p의 대가로 FPR +4.9%p
0.11	0.796	0.985	거의 다 생존 판정 → 98.5% 탐지하지만 사망자 79.6% 오판

■ 결과 해석

임계값이 낮아질수록 FPR·TPR이 **함께 단조 증가**합니다 — Positive 판정이 늘면 TP와 FP가 동시에 늘기 때문. 한쪽만 좋아질 수 없다는 트레이드오프가 배열에 그대로 보입니다. 좋은 임계값 후보는 "FPR이 조금 오르는 동안 TPR이 크게 뛰는 구간"의 끝(여기서는 0.66~0.43 부근)입니다. 이 (FPR, TPR) 쌍들을 좌표평면에 찍어 이은 것이 ROC 곡선, 그 아래 면적이 AUC입니다.

⚠ 버전 차이 주의 — thresholds[0]

thresholds[0]은 곡선이 (0,0)에서 시작하도록 넣은 **인위적 값**으로 실제 임계값이 아닙니다. scikit-learn 1.3 미만에서는 $\max(\text{예측확률})+1$, **1.3부터는 np.inf**로 변경되었습니다(실행 확인: thresholds[0] = inf, 최대 확률 0.965). 어느 버전이든 첫 원소를 제외하는 코드 의도(np.arange를 1부터)는 동일하게 유효합니다.

7.4 AUC 숫자 읽기

AUC 값	해석
0.5	무작위 추측과 동일(대각선) — 구분 능력 없음. 점수·정답 정렬표 전체가 교대 구간
0.6~0.7	구분 능력 약함
0.7~0.8	보통
0.8~0.9	좋음 (본 예제 0.8513이 여기 해당)
0.9~1.0	매우 좋음. 단 0.97 이상이면 데이터 누수(leakage) 의심
0.5 미만	예측이 뒤집혀 있음 — 레이블 정의·확률 열 선택 오류 확인 (9장 사례 5)

⚠ 자주 하는 실수

① roc_auc_score(y_test, pred)처럼 0/1 **판정값**을 넣으면 안 됩니다 — 반드시 predict_proba[:, 1]의 **확률**을 넣어야 합니다. ② AUC는 점수의 **순위**만 평가하므로, 점수의 절대값이 실제 비율과 맞는지(0.35점짜리들이 실제로 35%쯤 생존하는지)는 별도 주제인 확률 보정(calibration)입니다. ③ 극단적 불균형(양성 1% 등)에서는 TN이 많아 FPR이 잘 오르지 않으므로 ROC-AUC가 낙관적으로 나옵니다 — PR-AUC(average_precision_score) 병행 확인.

8. 단계별 해석 루틴 (실전 5단계)

모델 평가 결과를 받았을 때 지표를 읽는 순서입니다. 타이타닉 예제 값으로 각 단계를 시연합니다.

단계	할 일	보는 것	타이타닉 예제 적용
1	오차 행렬 확인	4개 칸의 절대 건수 . FP·FN의 위치와 규모, 클래스 불균형 정도	TN=94, FP=9, FN=21, TP=44 — 놓침(21)이 오탐(9)의 2배 이상

2	AUC로 체급 확인	점수가 두 클래스를 구분하는가. 낮으면 임계값 튜닝 무의미	0.8513 — 구분 능력 양호, 튜닝할 가치 있음
3	정밀도 vs 재현율 진단	모델이 어느 쪽으로 치우쳤는지. F1과 산술 평균의 차이로 불균형 정도 확인	0.8302 vs 0.6769 — 정밀도 쪽으로 기울 = 재현율 부족
4	업무 요구와 대조	FN 비용이 크다(재현율 우선), FP 비용이 크다(정밀도 우선)	생존자 탐지가 목적이라면 재현율 0.68은 부족 → 임계값 인하 검토
5	임계값 조정 후 재평가	precision_recall_curve 로 임계값별 곡선을 그려 균형점 선택, F1로 확인	7.3의 표에서 0.66~0.43 구간이 효율적 후보

■ 루틴의 논리

순서가 중요합니다. AUC(2단계)가 낮으면 3~5단계는 헛수고입니다 — 교대 구간 자체가 넓으면 어디를 잘라도 오류가 크기 때문입니다. 반대로 AUC가 높는데 F1이 낮으면 모델을 버릴 게 아니라 임계값만 옮기면 됩니다. **"모델의 문제인가, 임계값의 문제인가"**를 가르는 것이 이 루틴의 목적입니다.

9. 사례별 해석 가이드 (6가지 상황)

사례 1 — 정확도는 높는데 재현율·정밀도가 0에 가깝다

- **증상:** 정확도 0.90, 정밀도 0.0, 재현율 0.0 (2~4장의 가짜 분류기)
- **진단:** 불균형 데이터에서 다수 클래스로만 찍는 모델. 오차 행렬의 TP가 0인지 확인
- **처방:** 정확도를 평가 지표에서 제외하고 F1·AUC 중심으로 전환. 학습 시 `class_weight='balanced'`, 리샘플링 등 불균형 대응 검토

사례 2 — AUC는 높는데 F1이 낮다

- **증상:** AUC 0.9, F1 0.3. 예: 점수가 전부 0.5 미만에 몰려 있어(최고점 0.45) 임계값 0.5에서 전원 Negative 판정 → 재현율 0
- **진단:** 점수의 구분(정렬)은 완벽에 가까운데 **자르는 위치가 엉뚱한** 상태. "판별력 만점, 분류 성적 뺑뺑"
- **처방:** 모델 교체가 아니라 **임계값 튜닝**. precision_recall_curve 로 F1 최대 지점 탐색

사례 3 — AUC 자체가 낮다 (0.5 근처)

- **증상:** 임계값을 아무리 옮겨도 정밀도·재현율이 함께 낮음
- **진단:** 점수·정답 정렬표 전체가 교대 구간 — 양성 건과 음성 건이 **비슷한 피쳐값을 가져서 공식에 넣으면 비슷한 점수가 나오는** 상태. 현재 피쳐에 구분 정보가 없음
- **처방:** 임계값 튜닝으로는 해결 불가(오류의 총량은 교대 구간 크기가 결정). **피쳐 엔지니어링 또는 모델 변경**으로 점수 계산 자체를 고쳐야 함

사례 4 — 재현율은 1.0인데 정밀도가 바닥이다 (또는 그 반대)

- **증상:** 재현율 1.0, 정밀도 0.1 — 또는 정밀도 1.0, 재현율 0.02

- **진단:** 전부 Positive로 찍는 남발형(전자) 또는 확실한 1건만 찍는 소극형(후자). 한 지표만 높은 모델은 꿈수 가능성 우선 의심
- **처방:** F1로 균형을 확인하고, Positive 예측 건수(FP+TP)를 오차 행렬에서 직접 확인

사례 5 — AUC가 0.5 미만이다

- **증상:** AUC 0.3
- **진단:** 쌍 비교 승률 30% = 예측을 **반대로 뒤집으면 AUC 0.7**이라는 뜻. 모델이 무능한 게 아니라 방향이 뒤집힘 — 레이블 정의(0/1 의미)나 확률 열 선택([:, 0] 을 썼는지) 버그의 전형적 신호
- **처방:** classes_ 순서, 레이블 인코딩, predict_proba[:, 1] 사용 여부 점검

사례 6 — 극단적 불균형 데이터인데 ROC-AUC가 높게 나온다

- **증상:** 양성 1% 데이터에서 ROC-AUC 0.95인데 실제 오탐이 체감상 많음
- **진단:** TN이 압도적으로 많으면 FPR 분모가 커서 FPR이 잘 오르지 않음 → ROC-AUC가 낙관적
- **처방:** PR-AUC(average_precision_score)와 정밀도@재현율 구간을 함께 보고 판단 (신용카드 사기 검출 등)

10. 핵심 API 요약표

API	입력 (인자 순서 주의)	반환·용도
accuracy_score(y_true, y_pred)	실제, 0/1 판정값	정확도. 불균형 데이터에서는 단독 사용 금지
confusion_matrix(y_true, y_pred)	실제, 0/1 판정값	2×2 배열. 행=실제, 열=예측, 레이블 오름차순. labels 로 순서 지정, normalize 로 비율화 가능
precision_score(y_true, y_pred)	실제, 0/1 판정값	TP/(FP+TP). 0/0 상황은 zero_division 으로 제어
recall_score(y_true, y_pred)	실제, 0/1 판정값	TP/(FN+TP). 별칭: 민감도, TPR
f1_score(y_true, y_pred)	실제, 0/1 판정값	정밀도·재현율의 조화평균
roc_curve(y_true, y_score)	실제, Positive 확률	(fprs, tprs, thresholds) 3개 배열. 같은 인덱스끼리 한 세트. thresholds[0]은 인위값(1.3+ 버전: inf)
roc_auc_score(y_true, y_score)	실제, Positive 확률	ROC 곡선 아래 면적 = 쌍 비교 승률. 판정값을 넣으면 왜곡
precision_recall_curve(y_true, y_score)	실제, Positive 확률	임계값별 (정밀도, 재현율) — 임계값 튜닝의 기본 도구
average_precision_score(y_true, y_score)	실제, Positive 확률	PR-AUC. 극단 불균형에서 ROC-AUC 보완
모델.predict_proba(X)	피쳐	

		(n, 클래스수) 확률. 열 순서는 <code>classes_</code> (오름차순). Positive 확률은 <code>[:, 1]</code>
<code>모델.decision_function(X)</code>	피쳐	정규화 전 판별 점수(로짓). ROC 계산에 확률 대신 사용 가능
<code>ConfusionMatrixDisplay.from_predictions(y_true, y_pred)</code>	실제, 판정값	오차 행렬 시각화 (scikit-learn 1.0 이상 안정 API)

11. 머신러닝 활용 포인트

- **지표는 두 층으로 나눠 기억:** 예측 확률의 구분 능력(AUC — 임계값 무관, 모델의 체급) vs 판정의 성적표(정확도·정밀도·재현율·F1 — 임계값에 종속). 문제 진단의 첫 질문은 항상 "모델의 문제인가, 임계값의 문제인가"
- **불균형 데이터에서 정확도는 참고용:** 가짜 분류기도 90%가 나옵니다. 오차 행렬 → F1 → AUC 순으로 확인하는 습관
- **지표 선택은 오류 비용에서 출발:** FN이 치명적이면 재현율(암 진단·사기 탐지), FP가 치명적이면 정밀도(스팸 필터). 지표가 업무를 결정하는 게 아니라 업무가 지표를 결정
- **확률을 넣을 곳과 판정을 넣을 곳 구분:** `*_score(y, pred)` 계열은 0/1 판정값, `roc_auc_score · *_curve` 계열은 `predict_proba[:, 1]`. 혼용은 가장 흔한 버그
- **predict()는 predict_proba() + 임계값 0.5일 뿐:** 0.5를 성역으로 두지 말고 `precision_recall_curve` 로 업무에 맞는 임계값을 직접 고르기
- **AUC가 높다고 확률을 믿지는 말 것:** AUC는 순위만 평가. 확률의 절대값을 사용하는 업무(기대손실 계산 등)라면 확률 보정(calibration) 별도 확인
- **버전 확인 습관:** `roc_curve` 의 `thresholds[0]`(1.3부터 inf), `zero_division` (0.22부터) 등 버전에 따라 동작이 다른 부분은 실행 환경에서 직접 확인